API REST et JSON

CE QUE L'EXAMEN ATTEND — Décrire les API REST (authentification, CRUD, verbes HTTP, encodage des données) et reconnaître les composants d'un document JSON.

01 L'ESSENTIEL

01 REST : architecture client-serveur sans état (stateless : chaque requête porte tout le nécessaire), sur HTTP ou HTTPS ; ressources identifiées par des URI ; réponses éventuellement mises en cache.

02 CRUD et verbes HTTP : Create = POST, Read = GET, Update = PUT (remplacement) ou PATCH (modification partielle), Delete = DELETE.

03 Codes de retour : 2xx succès (200 OK, 201 Created, 204 No Content), 3xx redirection, 4xx erreur du client (400, 401 Unauthorized, 403 Forbidden, 404 Not Found), 5xx erreur du serveur.

04 Authentification : Basic (identifiant et mot de passe encodés en Base64, à réserver à HTTPS), clé d'API, jeton (bearer token), OAuth.

05 Encodage : JSON, XML ou YAML. En-tête `Content-Type` : format envoyé ; `Accept` : format attendu en réponse.

06 JSON : objet `{ }` de paires `"clé": valeur` séparées par des virgules ; tableau `[ ]` ; valeurs : chaîne entre guillemets doubles, nombre, `true` / `false`, `null`, objet, tableau. Pas de virgule finale, pas de commentaire.

```
GET /restconf/data/ietf-interfaces:interfaces/interface=GigabitEthernet1
Accept: application/yang-data+json → 200 OK

{
  "ietf-interfaces:interface": {                                  objet {}
    "name": "GigabitEthernet1",                                   clé : chaîne
    "description": "LAN",
    "enabled": true,                                              valeur booléenne
    "ietf-ip:ipv4": {
      "address": [                                                tableau []
        { "ip": "192.0.2.10", "netmask": "255.255.255.0" }
      ]
    }
  }
}
```

FIG. 39 Lecture (GET) d'une interface par RESTCONF ; le routeur répond 200 OK avec un document JSON conforme au modèle YANG ietf-interfaces.

02 COMMANDES CLÉS

COMMANDE	RÔLE
`$ curl -X GET https://192.0.2.10/restconf/data/ietf-interfaces:interfaces`	Read : lit les interfaces
`$ curl -X POST … -d @vlan.json`	Create : crée une ressource
`$ curl -X PUT … -d @intf.json`	Update : remplace la ressource
`$ curl -X PATCH … -d @desc.json`	Update partiel
`$ curl -X DELETE …`	Delete : supprime
`$ curl -u admin:Lab-Pass …`	Authentification Basic
`$ curl -H "Accept: application/yang-data+json" …`	Format de réponse attendu
`$ python3 -m json.tool reponse.json`	VÉRIF. Valide et indente un JSON

03 PIÈGES D'EXAMEN

P1 PUT remplace toute la ressource ; PATCH ne modifie que les champs fournis.

P2 401 : authentification absente ou invalide. 403 : identité reconnue, accès refusé.

P3 JSON : guillemets doubles obligatoires, aucune virgule après le dernier élément, true et false en minuscules.

04 AUTO-TEST

1 Quel verbe HTTP correspond à l'opération Create du modèle CRUD ?

A. GET **B.** POST **C.** PATCH **D.** DELETE

2 Quel code renvoie l'API quand la ressource demandée n'existe pas ?

A. 200 **B.** 401 **C.** 404 **D.** 500

3 Quel document JSON est valide ?

A. `{'vlan': 10}` **B.** `{"vlan": 10, "nom": "VOIX",}`

C. `{"vlan": 10, "actif": true}` **D.** `{vlan: 10}`

REPÈRE CRUD = POST, GET, PUT ou PATCH, DELETE.

RÉPONSES **1 B** Create = POST ; Read = GET ; Update = PUT ou PATCH ; Delete = DELETE.
2 C 404 Not Found : erreur côté client sur l'URI demandée.
3 C A : guillemets simples ; B : virgule finale ; D : clé sans guillemets.